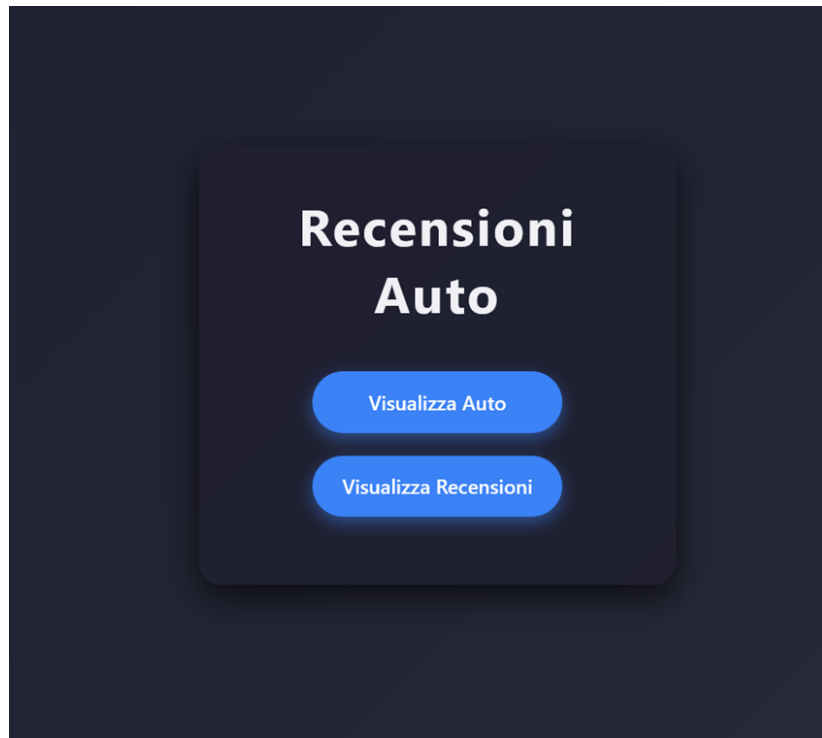


CAR-REVIEW: SISTEMA NOSQL PER LA GESTIONE E L'ANALISI DI RECENSIONI DI AUTOMOBILI

ANDREA SALURSO AND COSTANTINO PACIELLO



[Codice Sorgente Github](#)

Abstract. Negli ultimi anni, la crescita esponenziale dei dati generati dagli utenti ha reso necessaria l'adozione di soluzioni database flessibili e scalabili, come i sistemi No-SQL. In questo lavoro si presenta la progettazione e realizzazione di un sistema per la gestione e analisi delle recensioni di automobili, basato su MongoDB e sviluppato tramite una web application. Il sistema permette operazioni CRUD su auto e recensioni, offrendo un'interfaccia semplice per visualizzare, aggiungere, modificare o eliminare dati. Inoltre, la funzionalità di aggregazione consente di collegare informazioni tra auto e recensioni, facilitando una consultazione completa e interattiva delle opinioni degli utenti.

Contents

	2
1	Introduzione
2	Descrizione del Miniworld
2.1	Stato dell'arte
2.2	Pulizia e preparazione dei dati
2.3	Collezioni utilizzate
3	Contesto Applicativo
3.1	Estendibilità futura
4	Soluzione Proposta
4.1	Tecnologie Utilizzate
4.2	Architettura del sistema
4.3	Descrizione dettagliata delle funzionalità
4.4	Conformità alle proprietà BASE e CAP
5	Metodologia
5.1	Implementazione del Backend
5.2	Implementazione del Frontend
5.3	Strumenti e Tecniche
5.4	Validazione e Test

1 Introduzione

Negli ultimi anni, l'esplosione di dati generati dagli utenti nel contesto del web ha portato a una crescente adozione di database No-SQL per la gestione flessibile e scalabile delle informazioni. Questo progetto si inserisce in tale scenario, con l'obiettivo di progettare e realizzare un sistema per la gestione e analisi di recensioni di automobili da parte degli utenti. L'approccio adottato si basa sull'utilizzo di MongoDB, un DBMS orientato ai documenti, e sullo sviluppo di una web application con Flask per consentire operazioni CRUD e visualizzazione interattiva dei dati.

L'interfaccia dell'applicazione presenta una homepage semplice ed efficace, da cui l'utente può accedere a due principali sezioni: la visualizzazione delle auto e quella delle recensioni.

Nella sezione "**Visualizza auto**" vengono elencati tutti i modelli disponibili, ciascuno dei quali raggruppa le auto corrispondenti. Ogni singola auto può essere modificata o eliminata, e l'utente può anche aggiungere nuove auto alla collezione. Cliccando su un'auto, si accede a una pagina di dettaglio con tutte le sue informazioni e le recensioni associate, che possono anch'esse essere modificate o cancellate.

Parallelamente, dalla homepage è possibile accedere alla sezione "**Visualizza recensioni**", dove viene effettuata un'aggregazione tramite JOIN tra le collezioni di auto e recensioni. Qui, l'utente può selezionare una marca e visualizzare tutte le recensioni associate alle auto di quel brand, offrendo così una visione completa.

2 Descrizione del Miniworld

2.1 Stato dell'arte

I sistemi No-SQL sono diventati sempre più diffusi in applicazioni che richiedono scalabilità orizzontale, flessibilità dei dati e velocità nella gestione di grandi volumi. MongoDB è uno dei DBMS più utilizzati in ambito document-based, grazie alla sua struttura flessibile, basata su JSON, e al supporto per operazioni avanzate come le aggregazioni e l'indicizzazione. Nel contesto delle recensioni di prodotti o servizi, l'associazione tra entità e feedback degli utenti è un caso d'uso ideale per una struttura No-SQL.

2.2 Pulizia e preparazione dei dati

Il miniworld del progetto è stato costruito a partire da due dataset pubblici:

- Un dataset contenente modelli di auto, con attributi come marca, modello, anno, tipo di carburante, trasmissione, ecc.
- Un dataset con recensioni associate ai modelli di auto, con informazioni su autore, punteggio e contenuto del commento.

La prima fase ha richiesto un'attività di pulizia, trasformazione e integrazione dei dati, svolta interamente su Google Colab tramite l'utilizzo delle librerie Python come Pandas e NumPy.

In particolare, si è proceduto con la normalizzazione degli identificatori, introducendo un campo comune chiamato `id_car` che ha permesso di collegare logicamente i dati dei modelli di auto con le rispettive recensioni. Durante questo processo, sono stati eliminati i record duplicati, gestiti i valori nulli e armonizzati i formati testuali (es. marca, modello, anno), in modo da garantire coerenza tra le due fonti.

Una volta completata la pulizia, i dataset sono stati convertiti nel formato JSON, così da essere compatibili con MongoDB. I file ottenuti sono poi stati importati nel database NoSQL tramite `mongoimport`, generando due collezioni distinte ma semanticamente collegate: `cars` e `reviews`.

Questa fase ha posto le basi per una gestione dei dati più robusta e per l'implementazione corretta delle operazioni CRUD e delle query aggregate sul database

2.3 Collezioni utilizzate

Per la costruzione del miniworld sono stati selezionati due dataset provenienti da Kaggle, una piattaforma molto diffusa per la condivisione di dati in ambito data science e machine learning. Dopo un'attenta analisi delle opzioni disponibili, sono stati scelti i dataset che presentavano la maggiore completezza in termini di attributi e quantità di dati, in modo da garantire una base solida e realistica per le operazioni di modellazione e analisi successive.

- Collezione `cars`: contiene i documenti relativi ai modelli di auto.
LINK: <https://www.kaggle.com/datasets/CooperUnion/cardataset?resource=download>
- Collezione `reviews`: contiene le recensioni scritte dagli utenti, collegate tramite `id_car`.

LINK: <https://www.kaggle.com/datasets/ankkur13/edmundsconsumer-car-ratings-and-reviews>

3 Contesto Applicativo

L'applicazione CarReview nasce con l'obiettivo di offrire un'interfaccia semplice ed efficace per la consultazione e la gestione di dati relativi a modelli di automobili e alle recensioni degli utenti. Si rivolge sia a un pubblico generico, che desidera leggere o lasciare un commento su un'auto, sia a gestori di un portale o servizio che intendono monitorare l'andamento delle valutazioni.

Nel contesto attuale, in cui la reputazione online di un prodotto è fortemente influenzata dalle opinioni degli utenti, disporre di uno strumento per raccogliere, aggregare e analizzare recensioni è strategico. Il sistema è stato quindi pensato per fornire una visione aggregata e dettagliata dei modelli recensiti, mantenendo allo stesso tempo la possibilità di intervenire sui dati in modo diretto (creazione, modifica, cancellazione).

3.1 Estendibilità futura

Il progetto è stato strutturato con attenzione all'evoluzione futura. L'adozione di MongoDB consente infatti una notevole flessibilità nello schema dei dati, permettendo l'aggiunta di nuovi campi o collezioni senza compromettere la compatibilità. È possibile, ad esempio, ampliare il sistema con nuove entità come utenti registrati, listini prezzi aggiornati o concessionari.

Inoltre, grazie al supporto di MongoDB per operazioni di aggregazione complesse, è possibile integrare funzionalità analitiche avanzate, come la valutazione temporale dell'andamento delle recensioni o il confronto tra modelli simili. Il sistema si presta infine ad essere interconnesso con API esterne, ampliando ulteriormente il valore informativo delle recensioni archiviate.

4 Soluzione Proposta

4.1 Tecnologie Utilizzate

Il progetto fa leva su un insieme di tecnologie consolidate e complementari tra loro:

- **MongoDB** è stato scelto come database No-SQL per la sua flessibilità nella gestione di documenti JSON, la possibilità di aggregazione e la scalabilità orizzontale.
- **Flask** è stato adottato come micro-framework Python per la creazione del backend dell'applicazione web. La sua leggerezza e modularità si adattano perfettamente a un'applicazione prototipale come questa.
- **PyMongo** è la libreria utilizzata per connettere Flask a MongoDB, consentendo operazioni CRUD e JOIN in modo diretto.
- **HTML** e **CSS** sono stati impiegati per costruire l'interfaccia utente, con uno stile minimalista e orientato alla funzionalità.
- **Pandas** è stato impiegato in fase di pre-processing per analizzare e convertire i dataset iniziali in formato JSON.

4.2 Architettura del sistema

L'architettura si articola su tre livelli principali:

- **Livello dati:** MongoDB contiene le collezioni cars e reviews, con `id_car` come chiave semantica di collegamento.
- **Livello applicativo:** gestito da Flask, che implementa le logiche di routing, le operazioni CRUD e la logica per la visualizzazione e l'aggregazione dei dati.
- **Livello presentazione:** realizzato con HTML e CSS, consente all'utente di interagire con i dati attraverso una UI semplice e immediata.

4.3 Descrizione dettagliata delle funzionalità

Scenari d'uso principali: L'utente ha accesso a una *dashboard* interattiva dalla quale può eseguire diverse operazioni:

- **Visualizzare i modelli di auto disponibili**, comprensivi di tutte le specifiche, tramite un apposito pulsante presente nella Homepage denominato "VISUALIZZA AUTO".
- **Consultare la lista completa delle recensioni**, con la possibilità di modificarle o eliminarle direttamente dalla dashboard.
- **Inserire una nuova recensione** per un modello selezionato, specificando il punteggio e il contenuto testuale.
- **Visualizzare le recensioni relative a uno specifico modello di auto**, tramite il pulsante "VISUALIZZA RECENSIONI". Quest'ultima funzionalità prevede l'esecuzione di una `$lookup` su MongoDB per effettuare una *JOIN* tra le collezioni `cars` e `reviews`, ottenendo così una vista aggregata che combina le informazioni tecniche del modello con le opinioni

degli utenti.

```
db.reviews.aggregate([
  {
    $lookup: {
      from: "cars",
      localField: "car_id",
      foreignField: "id",
      as: "car_info"
    }
  }
])
```

4.4 Conformità alle proprietà BASE e CAP

Il sistema rispetta le proprietà **BASE** tipiche di MongoDB: la consistenza è eventuale, la disponibilità è elevata e il sistema è progettato per tollerare guasti. In riferimento al teorema **CAP**, la JOIN è stata realizzata ottimizzando per la Disponibilità (**A**) e la Tolleranza alla Partizione (**P**), accettando la perdita di Consistenza immediata (**C**) nel caso di aggiornamenti concorrenti.

5 Metodologia

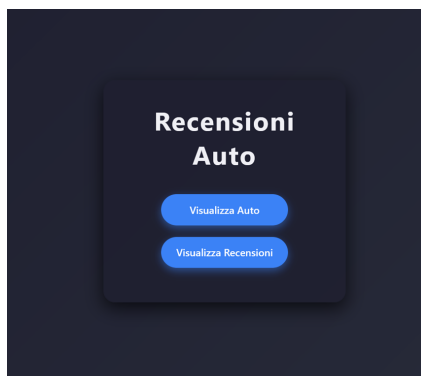
Lo sviluppo del progetto è stato affrontato con un approccio iterativo e incrementale, che ha permesso di affinare progressivamente sia l'architettura tecnica che l'esperienza utente. Ogni fase ha previsto momenti di progettazione, realizzazione e validazione, sempre con l'obiettivo di garantire funzionalità chiare, codice manutenibile e una base dati coerente.

5.1 Implementazione del Backend

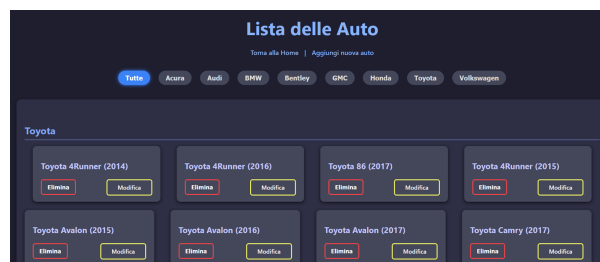
Il backend dell'applicazione è stato sviluppato in Python utilizzando il framework Flask. Attraverso l'uso della libreria PyMongo, è stata implementata una connessione efficiente con il database MongoDB, permettendo l'interazione con le collezioni cars e reviews. Ogni funzionalità CRUD è stata esposta tramite route RESTful, che consentono di inserire, modificare, eliminare e leggere documenti. Particolare attenzione è stata posta alla progettazione modulare delle route, alla validazione dei dati ricevuti in input e alla gestione degli errori. Ad esempio, l'inserimento di una nuova recensione avviene tramite una richiesta POST che, dopo aver validato i dati del form HTML, genera un nuovo documento JSON e lo inserisce nella collezione reviews. In caso di errore (dati mancanti o inconsistenza), il sistema fornisce un messaggio di risposta adeguato. L'operazione di JOIN tra le due collezioni è stata realizzata con l'uso della pipeline di aggregazione di MongoDB, sfruttando in particolare l'operatore \$lookup. Questa scelta ha permesso di combinare i dati relativi ai modelli con le relative recensioni, ottenendo una visione complessiva utile sia per la visualizzazione che per eventuali analisi future.

5.2 Implementazione del Frontend

L'interfaccia utente è stata progettata per essere il più semplice e funzionale possibile, mantenendo un look pulito e accessibile. Ogni pagina è strutturata in modo chiaro, con form dedicati all'inserimento di nuovi dati, tabelle per la visualizzazione e pulsanti per la modifica o eliminazione. La comunicazione tra frontend e backend avviene mediante richieste HTTP tradizionali, senza l'uso di JavaScript avanzato, privilegiando così la trasparenza e la facilità di debug. L'obiettivo non era costruire un'interfaccia grafica complessa, ma testare efficacemente tutte le funzionalità previste dal sistema.

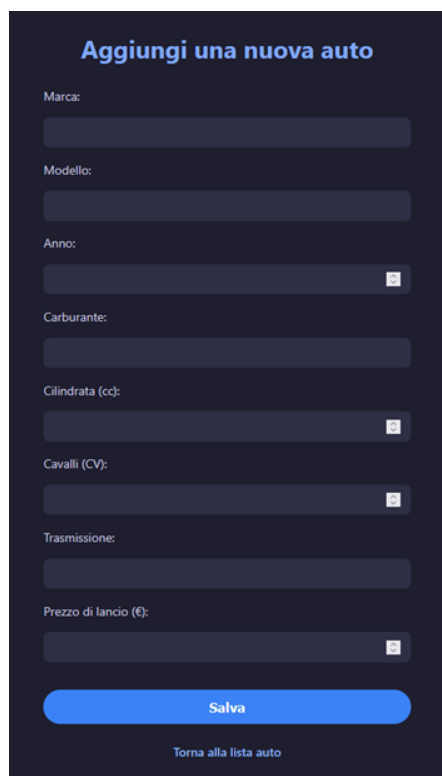


(A) HomePage

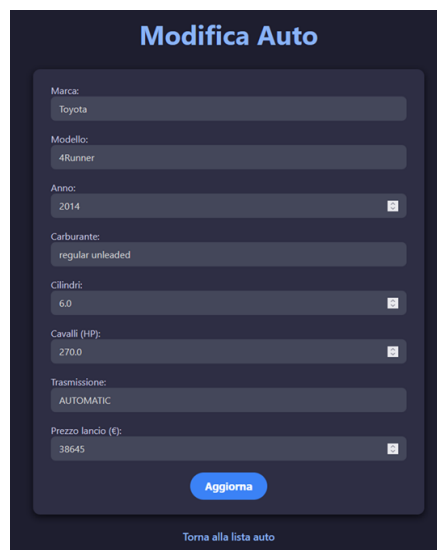


(B) Visualizzazione delle Auto

FIGURE 1. HomePage e visualizzazione modelli auto

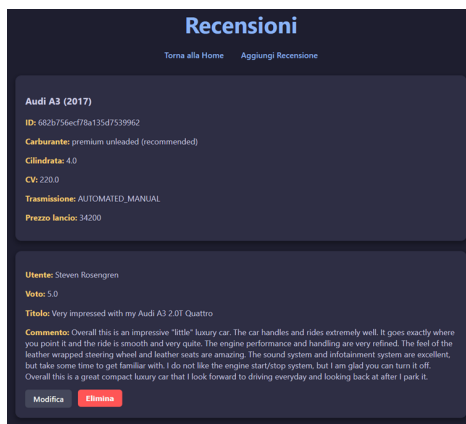


(A) Aggiunta nuova auto

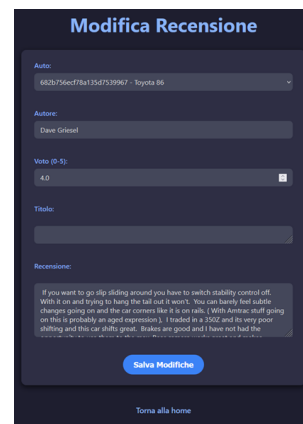


(B) Modifica dell'auto

FIGURE 2. Funzionalità di gestione auto



(A) Visualizza recensioni



(B) Modifica recensione

FIGURE 3. Gestione delle recensioni

5.3 Strumenti e Tecniche

Lo sviluppo è avvenuto in ambiente locale con Visual Studio Code come IDE principale e Git per il versionamento. I dati iniziali, in formato CSV, sono stati trattati con Pandas per rimuovere errori, duplicati e garantire coerenza strutturale prima della conversione in JSON.

Il database MongoDB è stato installato localmente, con collezioni create direttamente dall'applicazione o tramite script Python. Durante la fase di sviluppo, è stata attivata la modalità debug di Flask per velocizzare l'individuazione e la risoluzione di problemi.

5.4 Validazione e Test

Tutte le funzionalità sono state sottoposte a una serie di test manuali. Sono state verificate le operazioni CRUD su entrambe le collezioni, il corretto funzionamento della JOIN, e la coerenza tra i dati visibili nell'interfaccia e quelli archiviati nel database. Anche l'usabilità dell'interfaccia è stata valutata, correggendo eventuali ridondanze o problemi di navigazione.